

Young Innovators

Gesture-Controlled Robot Arm

Tilt your hand. The robot copies you.

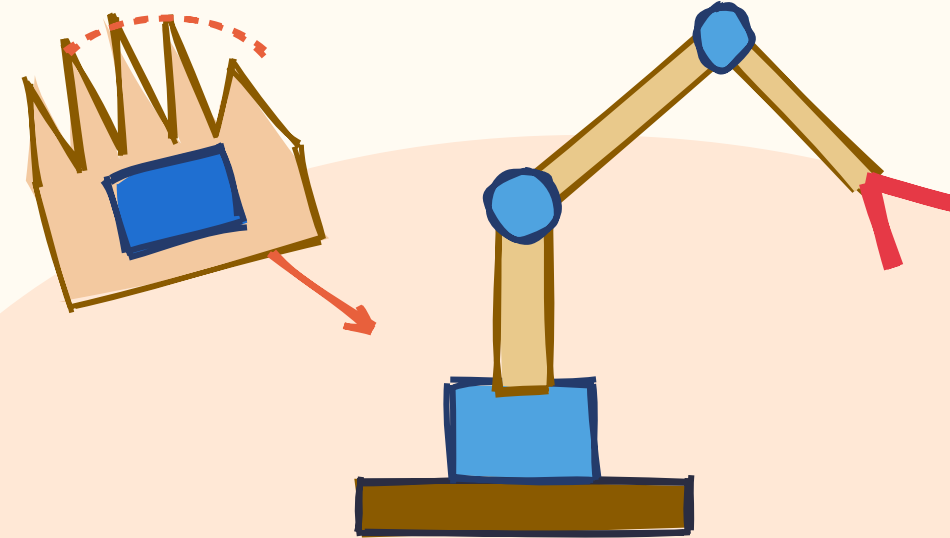
MPU6050 tilt sensing

Mapping ranges

Smoothing noisy data

Did you know?

Surgeons use robot arms that turn big hand movements into tiny, steady ones inside the body.





Let's think...

Too dangerous to touch?

Imagine

A suspicious bag is found at a railway station. Someone has to open it, but it might be dangerous.

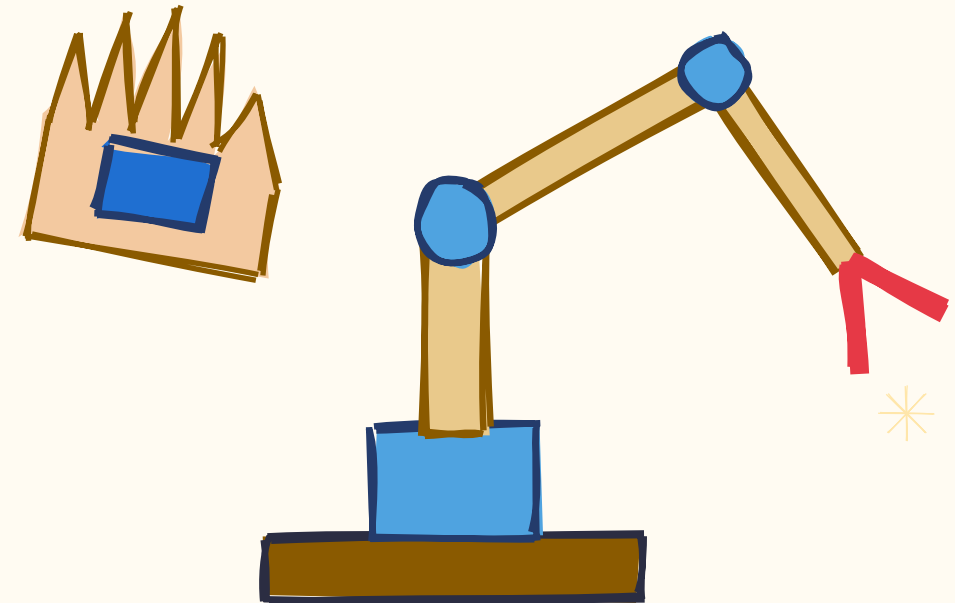
What if a robot arm could do it, copying the movements of an expert standing **50 metres away**?

Use a joystick?

Type commands?

Just move your hand!

Today: we build an arm that reads the **tilt of your hand** and copies it, live.





Learning aims

By the end of this session, I can...



1

Read tilt angles (**pitch** and **roll**) from an MPU6050

2

Use **map()** and **constrain()** to turn tilt into servo angles

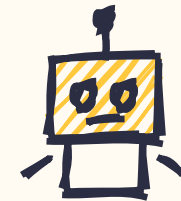
3

**Smooth** shaky sensor data with a moving average

4

Build a servo arm and explain **degrees of freedom**

5

Judge where **gesture control** is useful, and where it is not**Keywords**accelerometer · gyroscope · pitch ·
roll · map · smoothing · degrees of
freedom



Real world

Arms that follow people



Surgical robots

Scale down hand motion and filter out tremor



Bomb disposal

Experts work from a safe distance



Prosthetic hands

Read muscle and motion signals to move fingers



Your phone

The same sensor rotates the screen when you tilt it



Game controllers and VR headsets also use an **IMU** (inertial measurement unit) to track how you move.

Spot it

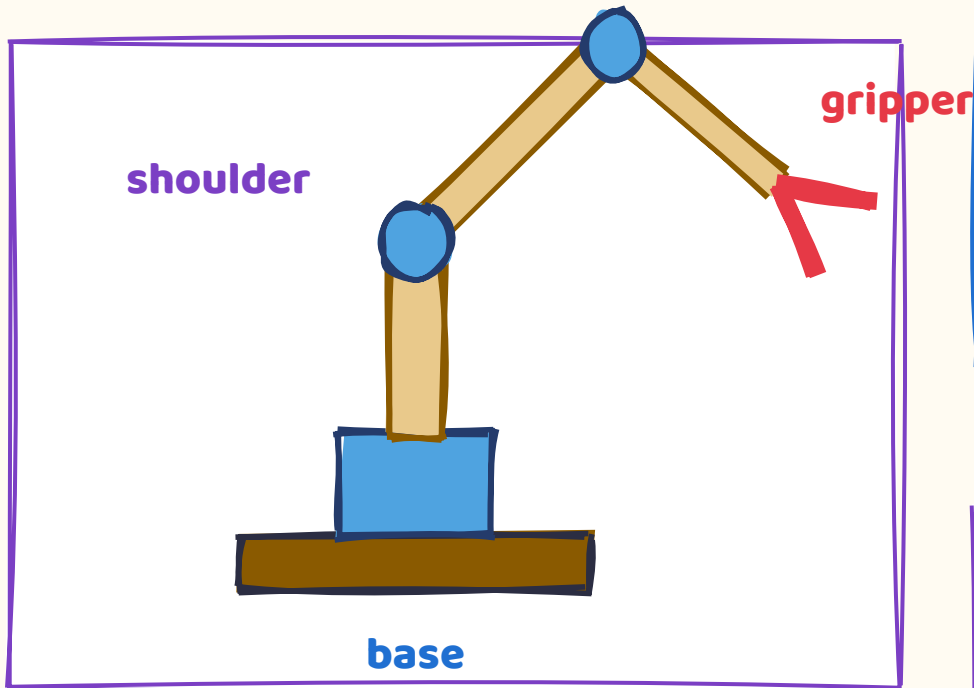
Turn your phone sideways. That is an accelerometer at work!





Engineering

Degrees of freedom (DOF)



What is a DOF?

Each joint that can move in **one independent way** is one degree of freedom. More DOF = more positions it can reach, but more motors and harder control.

Try it

Keep your shoulder still and only bend your elbow. How much less can you reach?

Your arm (shoulder to wrist)	7 DOF
Factory robot arm	6 DOF
Our arm	2 DOF + gripper

Our joints

Base turns left and right.
Shoulder lifts up and down.
Gripper opens and closes.



Stand up and play!

Mirror arm



How to play (pairs)

1. The **pilot** slowly tilts their hand left, right, forward, back.
2. The **robot arm** copies, but may only move **2 joints**: twist at the waist (base) and lift the arm (shoulder).
3. Pilot taps the table: robot closes its fist (gripper)!

What you discovered

Big tilt = big move: that is **mapping**.
Ignoring wobbles: that is **smoothing**.
Only 2 joints: that is **DOF**.



Shaky round: the pilot's hand trembles a tiny bit. Should the robot copy every wobble? What would happen to a surgeon's robot?

Notice

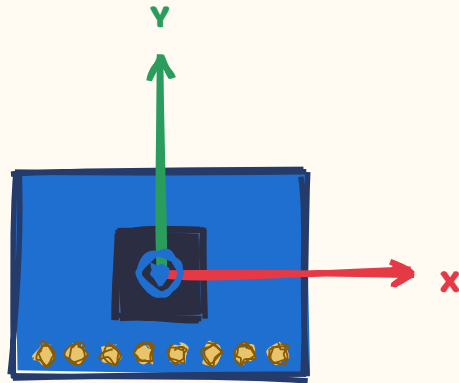
The robot reacts a tiny bit late. Smoothing always adds a little delay!





Meet the sensor

MPU6050: 6 sensors in one tiny chip



Accelerometer (3 axes)

Measures acceleration, **including gravity**. When the sensor is still, gravity tells us which way is down, so we can work out the **tilt**.

Fact file

Talks over **I2C** at address **0x68**.
Accelerometer range: **±2 g** by default.
1 g reads as **16384**.

Gyroscope (3 axes)

Measures how fast it is **rotating** (degrees per second). Great for fast moves, but it slowly **drifts** over time.

Tiny but clever

Inside is a microscopic weight on springs, carved from silicon (MEMS).

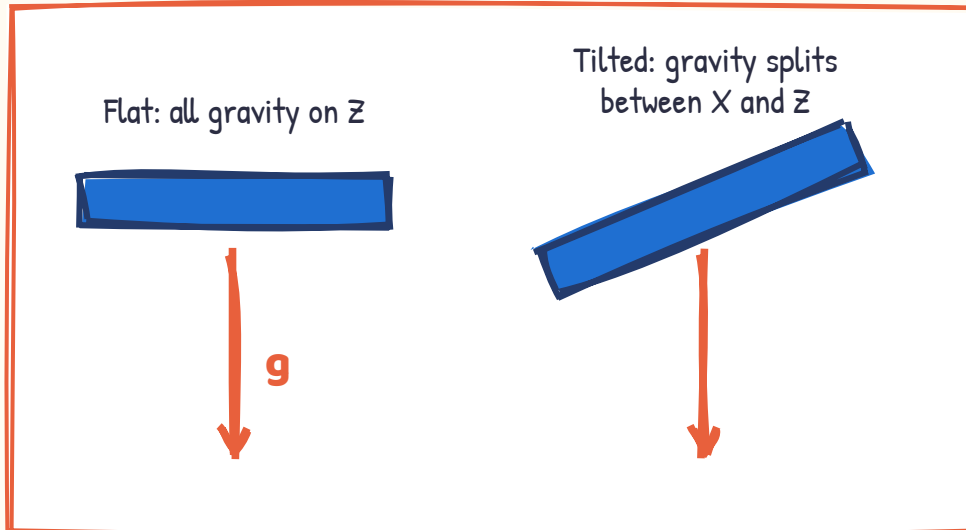


Pin	Connect to
VCC / GND	5V / GND
SCL	A5
SDA	A4
INT, AD0, XDA, XCL	Not needed today



The maths

Turning gravity into an angle



```
pitch = atan2(-ax, √(ay² + az²)) × 57.3
roll  = atan2(ay, az) × 57.3
× 57.3 turns radians into degrees (180 ÷ π)
```

Pitch and roll

Pitch: tip your hand forward or back (like nodding).

Roll: tip it left or right (like pouring tea).

The big limitation

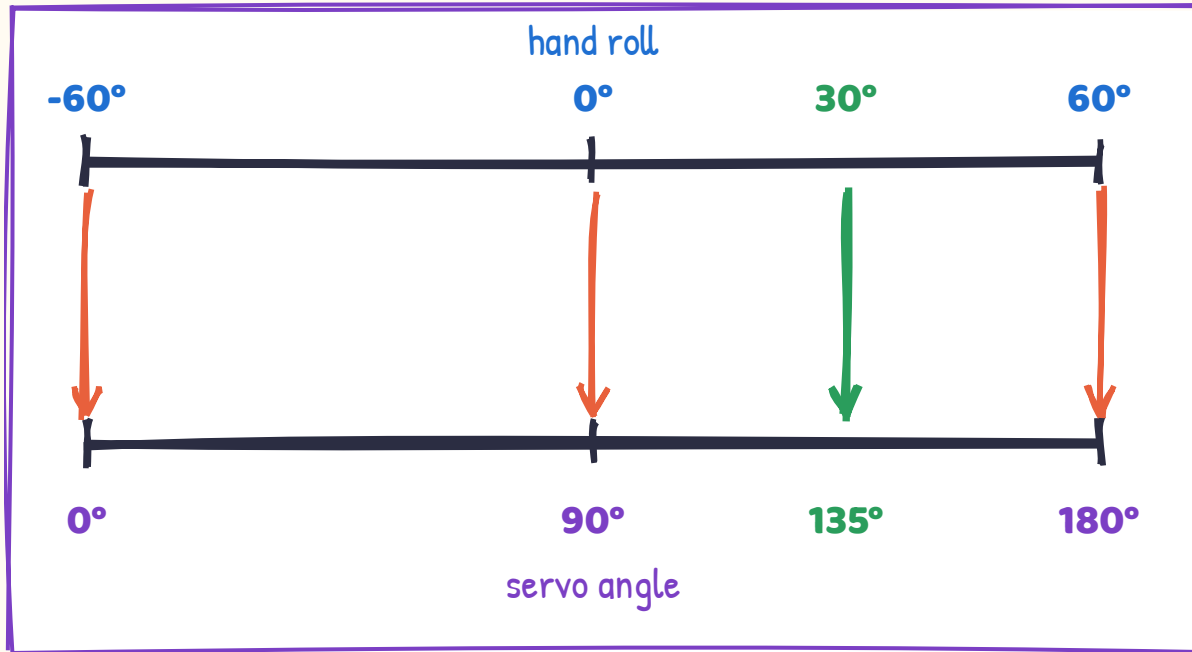
Turn your hand flat on the table like a steering wheel (**yaw**). Gravity does not change, so the accelerometer **cannot see it**. That is why our arm uses only pitch and roll.





Mapping

From hand angle to servo angle



```
int b = map(roll, -60, 60, 0, 180);
```

map(value, fromLow, fromHigh, toLow, toHigh)

Stretches one range onto another. A hand roll of 30° (three quarters of the way) becomes a servo angle of 135° .

constrain() first!

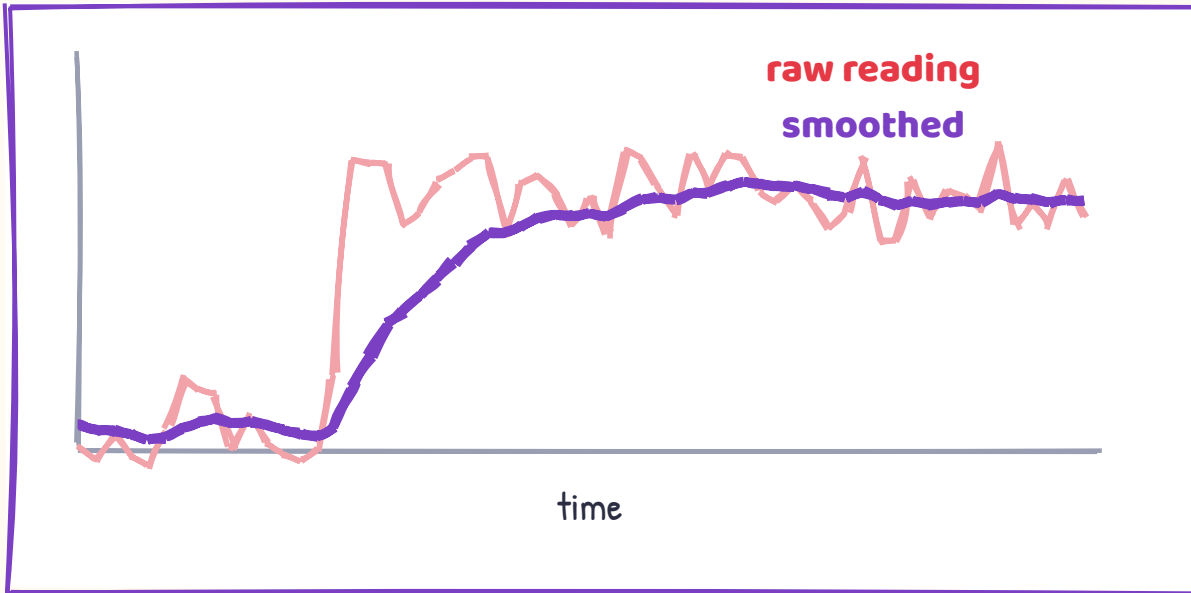
If you tilt to 80° , map() gives an angle **above 180**. Lock the input first:
`constrain(roll, -60, 60)`





Smoothing

Stop the arm from shaking



Exponential moving average

Each loop, take only **15%** of the new reading and keep **85%** of the old smooth value. Wobbles cancel out, real moves still come through.

```
smooth = 0.15 × new + 0.85 × smooth;
```

Alpha

Feels like

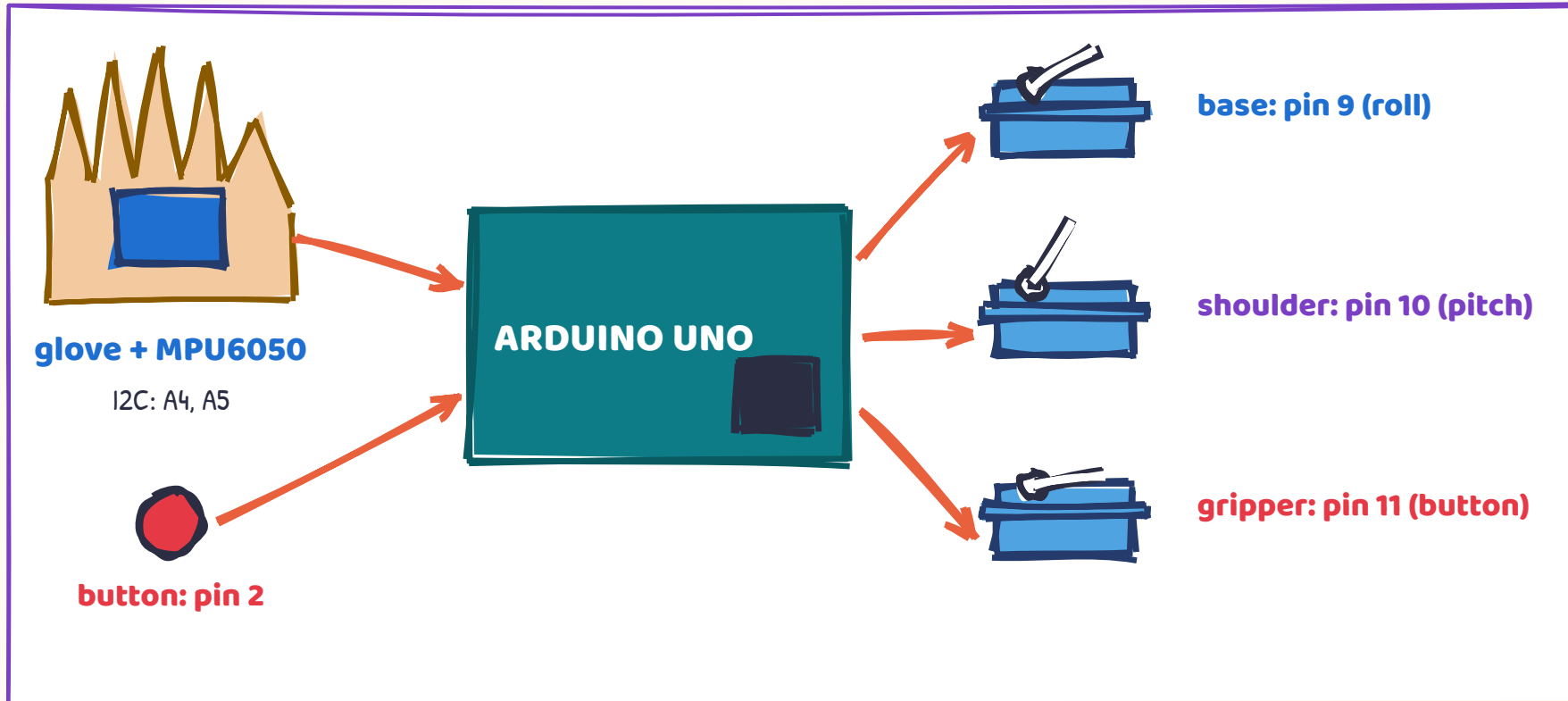
0.05	Super smooth, but slow and laggy
0.15	Good balance (our start)
0.5	Fast, but shaky





The plan

One glove, one brain, three servos



Loop, 50 times a second

Read tilt → smooth →
map → move servos →
check button.

Why a button?

Tilt only gives 2 angles. A button is a simple, reliable third control.

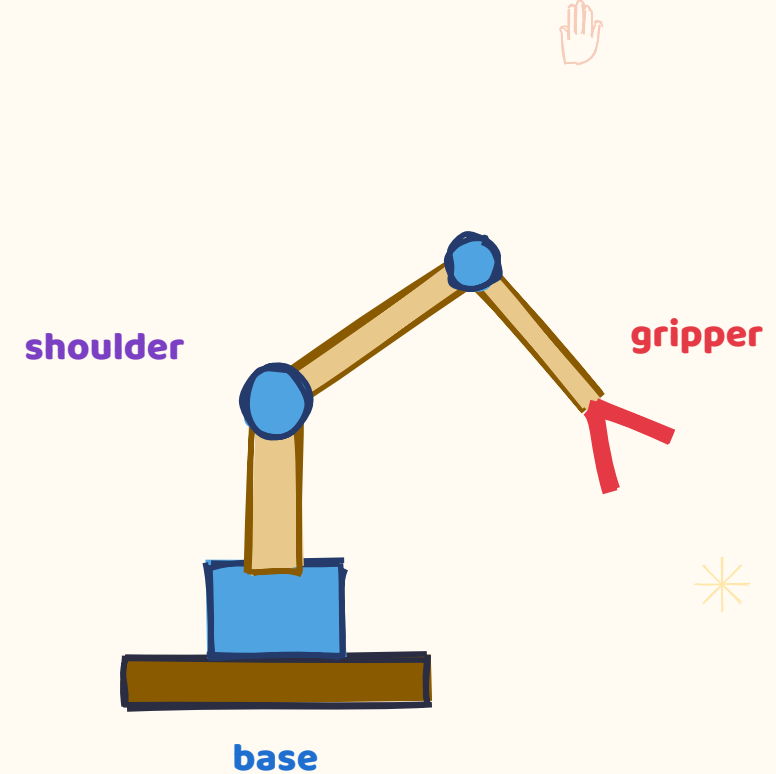




Build it

Make the arm

- 1 Base** Hot-glue (teacher) the base servo onto a heavy cardboard square, horn facing up.
- 2 Turntable** Glue a card disc on the base horn. Stand the shoulder servo on it.
- 3 Arm** Glue 2 or 3 ice cream sticks together for a stiff arm. Fix it to the shoulder horn.
- 4 Gripper** Glue one stick to the gripper horn, one fixed stick beside it: a pair of tweezers!
- 5 Glove** Tape or velcro the MPU6050 flat on the back of the glove, chip facing up.



Centre the servos first!
Upload a sketch that sets all servos to 90°, THEN attach the horns. Otherwise the arm points the wrong way.





Build it

Our robot arm kit (per group)

**Arduino Uno**

The brain

**MPU6050 (GY-521)**

Tilt sensor on the glove

**3 × SG90 servo**

Base, shoulder, gripper

**4×AA pack**

Servo power

**Ice cream sticks**

Arm structure

**Push button**

Gripper open / close

Also: an old glove, breadboard, long jumper wires (glove to breadboard), cardboard, glue gun (**teacher only**), a light object to pick up (a ping-pong ball or paper cup).

Long wires

Twist the 4 glove wires together so they do not snag when you move your hand.





Build it

Wiring and power



Connection	Goes to
MPU VCC / GND	5V / GND
MPU SDA / SCL	A4 / A5
Base servo signal	9
Shoulder servo signal	10
Gripper servo signal	11
Button	pin 2 and GND
All servos + / -	4×AA pack
Pack -	Arduino GND!

Why a battery pack?

Three servos moving at once can pull over **1 amp**. The Arduino's 5V pin cannot supply that: the board **resets** and the arm twitches. Give the servos their own **4×AA (6 V)** pack.

Max 6 V

SG90 servos are rated 4.8 to 6 V. Never use a 9 V battery on them!



Common ground

Connect the battery - to Arduino **GND**, or the servos will not understand the signal wires.

Button trick

INPUT_PULLUP means no resistor needed. Pressed = LOW.



Code it, part 1

Read the tilt (Arduino IDE)

```
#include <Wire.h>
#include <Servo.h>
Servo base, shoulder, grip;
const int MPU = 0x68;           // I2C address
const float A = 0.15;           // smoothing (0.05 to 0.5)
float pitchS = 0, rollS = 0;    // smoothed angles
bool gripOpen = true;

void setup() {
  Wire.begin();
  Wire.beginTransmission(MPU);
  Wire.write(0x6B); Wire.write(0); // wake the MPU6050 up
  Wire.endTransmission();
  base.attach(9); shoulder.attach(10); grip.attach(11);
  pinMode(2, INPUT_PULLUP);
}

void readTilt(float &pitch, float &roll) {
  Wire.beginTransmission(MPU);
  Wire.write(0x3B); // first accelerometer register
  Wire.endTransmission(false);
  Wire.requestFrom(MPU, 6);
  int16_t ax = Wire.read() << 8 | Wire.read();
  int16_t ay = Wire.read() << 8 | Wire.read();
  int16_t az = Wire.read() << 8 | Wire.read();
  pitch = atan2(-ax, sqrt((float)ay * ay + (float)az * az)) * 57.3;
  roll = atan2(ay, az) * 57.3;
}
```

Read the code

0x6B, 0: the MPU6050 starts asleep. This wakes it.

0x3B: where the 6 accelerometer bytes start.
<< 8 | joins two bytes into one 16-bit number.

float &pitch: the function fills in two answers at once.

No library needed

We talk to the chip directly. Now you know what libraries do for you!



Code it, part 2

Smooth, map, move

```
void loop() {  
  float pitch, roll;  
  readTilt(pitch, roll);  
  pitchS = A * pitch + (1 - A) * pitchS;    // smooth  
  rollS = A * roll + (1 - A) * rollS;  
  
  int b = map(constrain(rollS, -60, 60), -60, 60, 0, 180);  
  int s = map(constrain(pitchS, -60, 60), -60, 60, 30, 150);  
  base.write(b);  
  shoulder.write(s);  
  
  if (digitalRead(2) == LOW) {                // button pressed  
    gripOpen = !gripOpen;  
    grip.write(gripOpen ? 90 : 20);  
    delay(300);                                // simple debounce  
  }  
  delay(20);                                  // about 50 loops per second  
}
```



Why 30 to 150?

The shoulder should not slam into the table or flip over backwards. Limiting the servo range **protects the arm**. Find your own safe limits!



!gripOpen

! means NOT. Each press flips open to closed, or closed to open. This is called a **toggle**.

Try it

Change A to 0.05 and then 0.5. Feel the difference!





Test and debug

Is your arm under control?



Test	Expect	Result
Hand flat	Base and shoulder near 90°	pass / fail
Roll hand left / right	Base turns smoothly	pass / fail
Tip hand forward / back	Shoulder lifts and lowers	pass / fail
Shake hand a little	Arm stays steady	pass / fail
Press button	Gripper toggles	pass / fail

Debug checklist

Angles stuck at 0

Check SDA = A4, SCL = A5; run an I2C scanner

Arm moves the wrong way

Swap the to-range: map(..., 180, 0)

Arduino keeps resetting

Servos on battery pack, common GND

Gripper toggles twice

Increase the debounce delay



Print **pitchS** and **rollS** to the **Serial Plotter**. You will SEE the smoothing working.





Level up!

Pick-and-place challenge



Main challenge

Move a ping-pong ball (or paper cup) from **circle A** to **circle B** using only your glove. Fastest clean move wins!

Drop it = start again.

Discuss

Where is gesture control **better** than buttons?
Where is it **worse**? Think about a surgeon, a crane driver, a gamer.

More challenges



Freeze button: hold the button for 1 s to lock the arm in place.



4th servo: add an elbow joint. Which hand movement controls it?



Precision mode: map a small tilt ($\pm 20^\circ$) for fine, careful moves.



Boss level

Use the gyroscope to add yaw. Then explain why it drifts over time!



Show what you know

What did we learn?

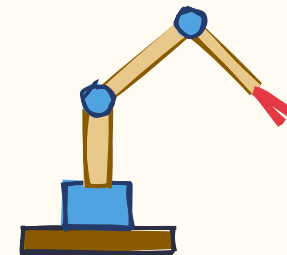
- ✓ An accelerometer feels gravity, so it can measure tilt
- ✓ Tilt gives pitch and roll, but not yaw
- ✓ `map()` + `constrain()` convert one range into another, safely
- ✓ Smoothing removes shake but adds a little delay
- ✓ Each independent joint is one degree of freedom



Exit ticket: roll is 15°. What is `map(15, -60, 60, 0, 180)`?

At home

Open a phone compass or level app. Tilt the phone and watch the angles change.



Next: an RFID Smart Door Lock with Access Log!